Department of

Artificial Intelligence & Machine Learning

B.Tech III Semester

Academic Year 2026-2027

Data Base Management Systems (25B11CS001)

Capstone Stone Project Documentation



ADITYA UNIVERSITY

Surampalem, Kakinada District

Andhra Pradesh-533437

ONLINE SHOPPING SYSTEM

A DBMS Capstone Project Report submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

A.N.V. Sai Baba - 25B11AI036

A. Santhosh - 25B11AI060

T. Bilva – 25B11AI133

T. Hemanth Kumar - 25B11AIC01

Under the esteemed guidance of

Mrs. G L Narasamba Vanguri, M. Tech, (Ph.D.)

Assistant Professor



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ADITYA UNIVERSITY

(Formerly Aditya Engineering College (A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

2026 – 2027

ADITYA UNIVERSITY
(Formerly Aditya Engineering College(A))
Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



CERTIFICATE

This is to certify that the DBMS Capstone Project report entitled "Online Shopping System" is being submitted by A.N.V.SAI BABA (Roll No. 25B11AI036), A. SANTHOSH (Roll No. 25B11AI060), T. BILVA (Roll No. 25B11AI133) and T. HEMANTH KUMAR (Roll No. 25B11AIC01) in the partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the **Capstone Project** for the course **Database Management Systems (2501IT05)** during the academic year **2026–2027**, and records the database design, implementation, and documentation undertaken by the team.

Project Guide
Mrs. G L Narasamba Vanguri, M.Tech, (Ph.D.)
Assistant professor

Head of the Department
Dr. K. Naga Bhargavi, M.Tech, Ph.D.
Associate Professor & HOD

External Examiner

DECLARATION

We hereby declare that the DBMS Capstone Project entitled "Online Shopping System" is a genuine work carried out by us. The database design, SQL implementation, queries and documentation presented in this report are our own work, completed under the guidance of Mrs. G L Narasamba Vanguri, Department of Artificial Intelligence and Machine Learning, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associates

A.N.V.SAI BABA (25B11AI036)

A.SANTHOSH (25B11AI060)

T. BILVA (25B11AI133)

T. HEMANTH KUMAR (25B11AIC01)

Date: 14 September 2026
Place: Surampalem

ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide Mrs. G L Narasamba Vanguri Department of Artificial Intelligence and Machine Learning, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to Dr. K. Naga Bhargavi, Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We extend our sincere thanks to the Management, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, Dr. M. B. Srinivas, Vice-Chancellor, and Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), for providing an excellent academic environment.

Not to forget, we thank the Teaching and Non-Teaching Staff, our parents and our friends, who have directly or indirectly supported us in completing this project on time.

Project Associates

A.N.V.SAI BABA (25B11AI036)

A. SANTHOSH (25B11AI060)

T. BILVA (25B11AI133)

T. HEMANTH KUMAR (25B11AIC01)

ADITYA UNIVERSITY

Surampalem

Department of Computer Science and Engineering

DBMS CAPSTONE PROJECT

Project Documentation Report

Project Title: Online Shopping System

Submitted by:

S.No	Student Name	Roll Number
1	A.N.V.SAI BABA	25B11AI1
2	A.SANTHOSH	25B11AI060
3	T.BILVA	25B11AI133
4	T.HEANTH KUMAR	25B11AIC01

Course: Database Management System (DBMS) Course Code: DBMS

Faculty Coordinator:

Academic Year: 2026–27 Date of Submission: 14 September 2026

GitHub Repository Link: https://github.com/DBMS-2026/Online-Shopping-System-DBMS

1. ABSTRACT

The **Online Shopping System** is a database-driven application designed to provide a simple, efficient, and user-friendly platform for purchasing products online. The system allows customers to browse products based on categories, add products to a shopping cart, place orders, make payments, and track deliveries. It also maintains important information such as customer details, product details, order information, payment records, and delivery status in a structured database.

The system is developed using **DBMS concepts such as relational database design, primary keys, foreign keys, constraints, CRUD operations, and SQL queries** to ensure data consistency and integrity. The database consists of major entities such as **Customer, Category, Product, Cart, Orders, Order Item, Payment, and Delivery**, with relationships established between them. This system reduces manual work, organizes shopping-related data efficiently, and provides faster access to information. Overall, the Online Shopping System demonstrates how database management concepts can be applied to develop a practical e-commerce application.

TABLE OF CONTENTS

Chapter	Title	Page No.
	Certificate	i
	Declaration	ii
	Acknowledgement	iii
	Project Documentation Report	iv
1	**ABSTRACT**	v
	List of Figures	vii
	List of Tables	vii
2	**INTRODUCTION**	1
	2.1 Background	1
	2.2 Purpose of the Database	1
3	**PROBLEM STATEMENT**	1
4	**OBJECTIVES**	1
5	**TOOLS & TECHNOLOGIES USED**	2
6	**DATABASE DESIGN**	2
	6.1 Identified Entities	2
	6.2 Entity Attributes	2
	6.3 ER Diagram	4
	6.4 Relationships & Cardinality	4
	6.5 Relational Database Schema	4
	6.6 Normalization	6
7	**DATABASE IMPLEMENTATION**	8
	7.1 DDL — Database and Table Creation	8
	7.2 Sample Data — DML.	10
	7.3 Constraints & Data Integrity	10
	7.4 SQL Operations & Queries	10
	7.5 Sample Outputs / Screenshots	12
8	**MODULES & FUNCTIONALITIES**	14
9	**GITHUB REPOSITORY & TEAM RESPONSIBILITIES**	14
10	**ENHANCEMENTS & FUTURE IMPROVEMENTS**	15
11	**CONCLUSION**	15
12	**REFERENCES**	15
	APPENDIX	16
	A. Complete SQL Script	16
	B. Additional Screenshots / Outputs	16

LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
6.3	ER Diagram	4
Figure 1	Customer records / basic SELECT output	12
Figure 2	filtering output	12
Figure 3	Aggregate or GROUP BY output	13
Figure 4	Join or subquery output	13

LIST OF TABLES

Table No.	Title of the Table	Page No.
5	TOOLS & TECHNOLOGIES USED	2
6.1	Identified Entities	2
6.2	Entity Attributes	2
6.4	Relationships & Cardinality	4
6.5	Relational Database Schema	4
7.2	Sample Data — DML.	10
7.3	Constraints & Data Integrity	10
7.4	SQL Operations & Queries	10
8	MODULES & FUNCTIONALITIES	14
9	GITHUB REPOSITORY & TEAM RESPONSIBILITIES	14

2. INTRODUCTION

2.1 Background

Online shopping systems handle large amounts of structured information related to customers, categories, products, carts, orders, payments, and deliveries. A relational database is required to store this information consistently and to maintain relationships between the different records. In this project, the online shopping domain is represented using eight related tables: **Customer, Category, Product, Cart, Orders, Order_Item, Payment, and Delivery**. The schema separates data into appropriate entities and uses primary keys, foreign keys, and constraints to reduce duplication and maintain data integrity. SQL queries are then used to retrieve, update, insert, delete, aggregate, and analyze the stored information.

2.2 Purpose of the Database

The database provides a centralized structure for managing **customer details, product information, category details, shopping carts, orders, order items, payment records, and delivery information**. It can support customers, shopping administrators, and store managers who need to browse products, manage carts, place orders, process payments, track deliveries, and maintain product and customer records. The relationships between the tables allow related information to be accessed through **SQL joins and subqueries** instead of maintaining disconnected records. This ensures organized data management, reduces redundancy, and helps maintain **data consistency and integrity** throughout the online shopping system.

3. PROBLEM STATEMENT

Traditional shopping systems and poorly organized online shopping platforms often face difficulties in managing large amounts of information related to **customers, products, categories, carts, orders, payments, and deliveries**. Maintaining these records separately can lead to data duplication, inconsistency, difficulty in tracking orders, and errors in managing customer and product information.

The system maintains relationships between **Customer, Category, Product, Cart, Orders, Order_Item, Payment, and Delivery** tables using primary keys, foreign keys, and constraints. SQL operations are used to perform **data insertion, retrieval, updating, deletion, and analysis**. The proposed system helps organize shopping data, maintain data integrity, reduce redundancy, and provide efficient access to information required for managing online shopping activities.

4. OBJECTIVES

- Objective 1: Design a relational schema containing eight related online shopping entities: **Customer, Category, Product, Cart, Orders, Order_Item, Payment, and Delivery**.
- Objective 2: Implement **primary keys, foreign keys, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints** to maintain data integrity and consistency.
- Objective 3: Populate the database with representative **customer, category, product, cart, order, order item, payment, and delivery records**.
- Objective 4: Demonstrate SQL operations including **data retrieval, insertion, updates, joins, aggregate functions, GROUP BY/HAVING, and subqueries**.
- Objective 5: Document the **database design, ER diagram, relational schema, implementation, sample data, SQL queries, and team responsibilities**.

5. TOOLS & TECHNOLOGIES USED

Category	Tool / Technology Used
Database Server	SQL
Query / Admin Tool	MySQL Workbench / MySQL Client
Front-end (if any)	Not implemented; database-focused project
Operating System	Windows
Version Control	GitHub

6. DATABASE DESIGN

6.1 Identified Entities

S.No	Entity Name	Description
1	Customer	Stores customer identification, contact details, address, and account-related information.
2	Category	Stores product categories and their descriptions for organizing products.
3	Product	Stores product details such as product name, price, stock, description, and category.
4	Cart	Stores shopping cart information associated with customers, including cart ID and creation date.
5	**Orders**	Stores customer order details such as order date, total amount, and order status.
6	**Order_item**	Stores the products included in each order along with quantity and price information
7	Payment	Stores payment details associated with orders, including payment date, method, status, and amount.
8	Delivery	Stores delivery information such as delivery address, delivery date, and delivery status.

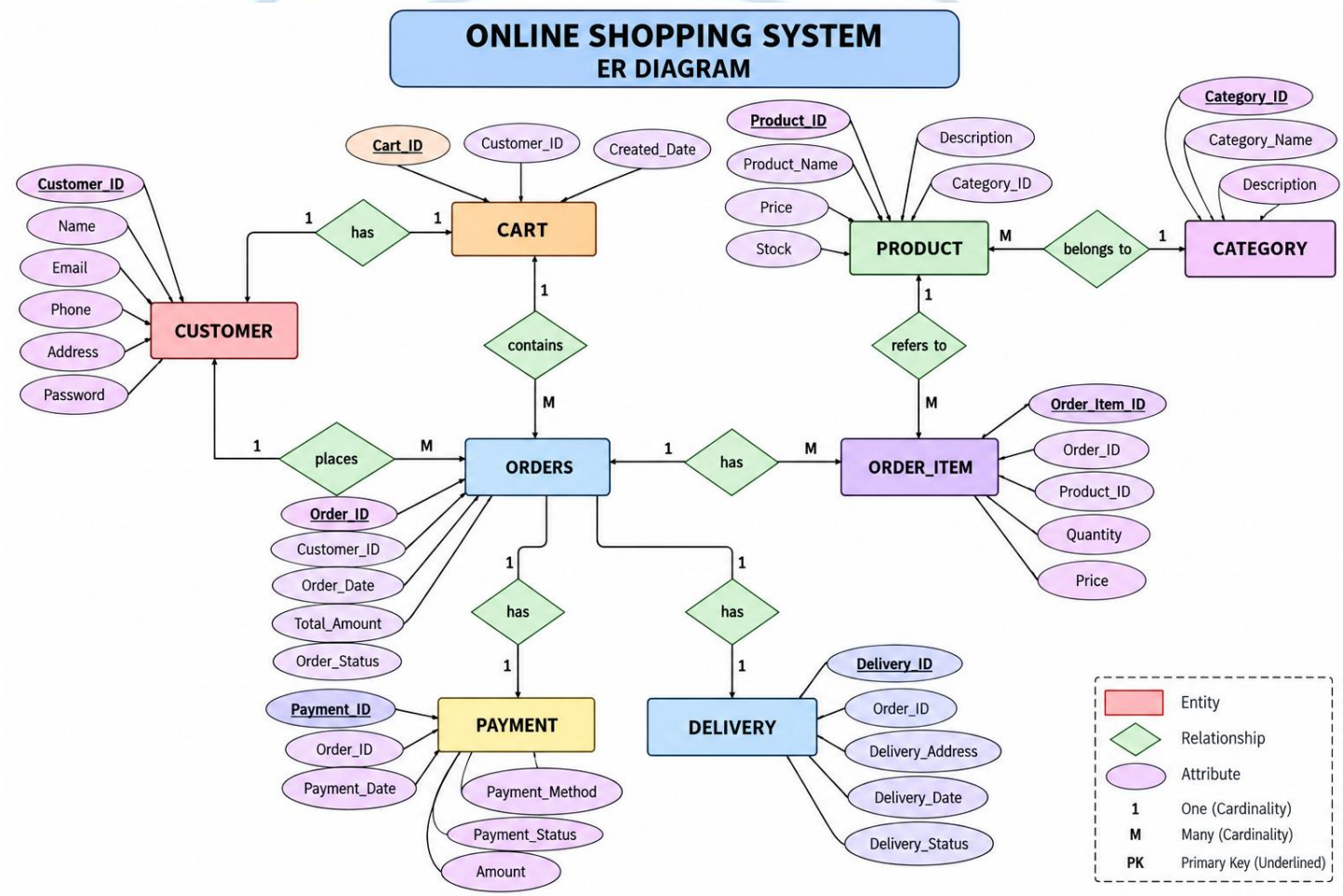
6.2 Entity Attributes

Entity	Attribute	Data Type	Key (PK/FK/—)	Attribute Type
Customer	Customer_ID	INTEGER	PK	Simple
Customer	Name	VARCHAR(100)	—	Simple
Customer	Email	VARCHAR(100)	—	Simple
Customer	Phone	VARCHAR(15)	—	Simple
Customer	Address	VARCHAR(255)	—	Simple
Customer	Password	VARCHAR(100)	—	Simple
Category	Category_ID	INTEGER	PK	Simple
Category	Category_Name	VARCHAR(100)	—	Simple
Category	Description	VARCHAR(255)	—	Simple
Product	Product_ID	INTEGER	PK	Simple

Product	Product_Name	VARCHAR(100)	—	Simple
Product	Price	DECIMAL(10,2)	—	Simple
Product	Stock	INTEGER	—	Simple
Product	Description	VARCHAR(255)	—	Simple

Entity	Attribute	Data Type	Key (PK/FK/—)	Attribute Type
Product	Category_ID	INTEGER	FK	Simple
Cart	Cart_ID	INTEGER	PK	Simple
Cart	Customer_ID	INTEGER	FK	Simple
Cart	Created_Date	DATE	—	Derived/Default
Orders	Order_ID	INTEGER	—	simple
Orders	Customer_ID	INTEGER	PK	Simple
Orders	Order_Date	DATE	FK	Simple
Orders	Total_Amount	DECIMAL(10,2)	—	Simple
Orders	Order_Status	VARCHAR(30)	—	Simple
Order_Item	Order_Item_ID	INTEGER	PK	Simple
Order_Item	Order_ID	INTEGER	FK	Simple
Order_Item	Product_ID	INTEGER	FK	Simple
Order_Item	Quantity	INTEGER	—	Simple
Order_Item	Price	DECIMAL(10,2)	—	Simple
Payment	Payment_ID	INTEGER	PK	Simple
Payment	Order_ID	INTEGER	FK	Simple
Payment	Payment_Date	DATE	—	Simple
Payment	Payment_Method	VARCHAR(30)	—	Simple
Payment	Payment_Status	VARCHAR(30)	—	Simple
Payment	Amount	DECIMAL(10,2)	—	Simple
Delivery	Delivery_ID	INTEGER	PK	Simple
Delivery	Order_ID	INTEGER	FK	Simple
Delivery	Delivery_Address	VARCHAR(255)	—	Simple
Delivery	Delivery_Date	DATE	—	Simple
Delivery	Delivery_Status	VARCHAR(30)	—	Simple

6.3 ER Diagram



6.4 Relationships & Cardinality

Entity 1	Relationship	Entity 2	Cardinality (1:1 / 1:N / M:N)
Orders	contains	Order_item	1:N
Orders	has	Payment	1:1
Category	Contains	Product	1:N
Customer	has	Cart	1:N
Customer	places	Orders	1:N
Product	Appears in	Order_Item	1:N
Orders	has	Delivery	1:1

6.5 Relational Database Schema

Table Name	Column Name	Data Type	PK/FK/—	Constraints
Customer	Customer_ID	INTEGER	PK	NOT NULL
Customer	Name	VARCHAR(100)	—	NOT NULL
Customer	Email	VARCHAR(100)	—	NOT NULL, UNIQUE
Customer	Phone	VARCHAR(15)	—	NOT NULL
Customer	Address	VARCHAR(255)	—	NOT NULL
Customer	Password	VARCHAR(100)	—	NOT NULL
Category	Category_ID	INTEGER	pk	NOT NULL
Category	Customer_Name	VARCHAR(100)	—	NOT NULL, UNIQUE
Category	Description	VARCHAR(255)	—	NULL,ALLOWED
Product	Product_ID	INTEGER	pk	NULL allowed
Product	Product_Name	VARCHAR(100)	—	NOT NULL
Product	Price	DECIMAL(10,2)	—	NOT NULL
Product	Stock	INTEGER	—	CHECK Male/Female/Other
Product	Description	VARCHAR(255)	—	NOT NULL
Product	Category_ID	INTEGER	FK	REFERENCES Category
Cart	Cart_ID	INTEGER	PK	NOT NULL
Cart	Customer_ID	INTEGER	FK	NOT NULL
Cart	Created_Date	DATE	—	DEFAULT CURRENT_DATE
Orders	Order_ID	INTEGER	PK	NOT NULL
Orders	Customer_ID	INTEGER	FK	REFERENCES Customer
Orders	Order_Date	DATE	—	DEFAULT CURRENT_DATE
Orders	Total_Amount	DECIMAL(10,2)	—	NOT NULL,CHECK >= 0
Orders	Order_Status	VARCHAR(30)	—	DEFAULT 'Placed’
Order_Item	Order_Item_ID	INTEGER	PK	NOT NULL
Order_Item	Order_ID	INTEGER	FK	NOT NULL, REFERENCES

				Orders
Order_Item	Product_ID	INTEGER	FK	NOT NULL, REFERENCES Product

Table Name	Column Name	Data Type	PK/FK/—	Constraints
Order_Item	Quantity	INTEGER	—	NOT NULL, CHECK > 0
Order_Item	Price	DECIMAL(10,2)	—	NOT NULL,CHECK > 0
Payment	Payment_ID	INTEGER	PK	NOT NULL
Payment	Order_ID	INTEGER	FK	NOT NULL, REFERENCES Orders
Payment	Payment_Date	DATE	—	DEFAULT CURRENT_DATE
Payment	Payment_Method	VARCHAR(30)	—	NOT NULL
Payment	Payment_Status	VARCHAR(30)	—	DEFAULT 'Pending
Payment	Amount	DECIMAL(10,2)	—	NOT NULL, CHECK > 0
Delivery	Delivery_ID	INTEGER	PK	NOT NULL
Delivery	Order_ID	INTEGER	FK	NOT NULL, REFERENCES Orders
Delivery	Delivery_Address	VARCHAR(255)	—	NOT NUL
Delivery	Delivery_Date	DATE	—	NULL allowed
Delivery	Delivery_Status	VARCHAR(30)	—	DEFAULT 'Pending'

6.6 Normalization

UNF → 1NF → 2NF → 3NF

UNF: Shopping information could initially be represented as a large combined record containing customer, category, product, cart, order, payment and delivery details.

1NF: The design keeps attributes atomic and separates repeating groups. Product, customer and order-item details are stored in separate rows and columns.

2NF: Each entity has a primary key, and non-key attributes depend on the complete key of their respective table. Order-item information is separated from Orders.

3NF: Relationships are represented using foreign keys. Category information is stored separately from Product, while Payment and Delivery information are stored separately from Orders. This reduces redundancy and update, insertion and deletion anomalies

7. DATABASE IMPLEMENTATION

7.1 DDL — Database and Table Creation

```
   -- Drop tables if they exist --
DROP TABLE IF EXISTS Delivery;
DROP TABLE IF EXISTS Payment;
DROP TABLE IF EXISTS
Order_Item;
DROP TABLE IF EXISTS Cart;
DROP TABLE IF EXISTS Product;
DROP TABLE IF EXISTS
category;
 DROP TABLE IF EXISTS CUSTOMER;
```

-- 1. CATEGORY TABLE:

```
CREATE TABLE CATEGORY (
    Category_ID INT PRIMARY KEY,
    Category_Name VARCHAR(100) NOT NULL UNIQUE,
    Description VARCHAR(255)
    );
```

-- 2. CUSTOMER TABLE:

```
CREATE TABLE Customer (
    Customer_ID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Phone VARCHAR(15) NOT NULL,
    Address VARCHAR(255) NOT NULL,
    Password VARCHAR(100) NOT NULL
    );
```

-- 3. PRODUCT TABLE:

```
CREATE TABLE PRODUCT (
    Product_ID INT PRIMARY KEY,
    Product_Name VARCHAR(100) NOT NULL,
    Price DECIMAL(10,2) NOT NULL CHECK (Price > 0),
    Stock INT NOT NULL CHECK (Stock >= 0),
    Description VARCHAR(255),
    Category_ID INT NOT NULL,
    FOREIGN KEY (Category_ID) REFERENCES CATEGORY(Category_ID)
    );
```

-- 4. CART TABLE:

```
CREATE TABLE CART (
    Cart_ID INT PRIMARY KEY,
    Customer_ID INT NOT NULL,
    Created_Date DATE DEFAULT CURRENT_DATE,
    FOREIGN KEY (Customer_ID) REFERENCES CUSTOMER(Customer_ID)
    );
```

```
-- 5. ORDERS TABLE:

CREATE TABLE ORDERS (
    Order_ID INT PRIMARY KEY,
    Customer_ID INT NOT NULL,
    Order_Date DATE DEFAULT CURRENT_DATE,
    Total_Amount DECIMAL(10,2) NOT NULL CHECK (Total_Amount >= 0),
    Order_Status VARCHAR(30) DEFAULT 'Placed',
    FOREIGN KEY (Customer_ID) REFERENCES CUSTOMER(Customer_ID)
    );

-- 6. ORDER ITEM TABLE:

CREATE TABLE ORDER_ITEM (
    Order_Item_ID INT PRIMARY KEY,
    Order_ID INT NOT NULL,
    Product_ID INT NOT NULL,
    Quantity INT NOT NULL CHECK (Quantity > 0),
    Price DECIMAL(10,2) NOT NULL CHECK (Price > 0),
    FOREIGN KEY (Order_ID) REFERENCES ORDERS(Order_ID),
    FOREIGN KEY (Product_ID) REFERENCES PRODUCT(Product_ID)
    );

-- 7. PAYMENT TABLE:

CREATE TABLE PAYMENT (
    Payment_ID INT PRIMARY KEY,
    Order_ID INT NOT NULL,
    Payment_Date DATE DEFAULT CURRENT_DATE,
    Payment_Method VARCHAR(30) NOT NULL,
    Payment_Status VARCHAR(30) DEFAULT 'Pending',
    Amount DECIMAL(10,2) NOT NULL CHECK (Amount > 0),
    FOREIGN KEY (Order_ID) REFERENCES ORDERS(Order_ID)
    );


-- 8. PAYMENT TABLE:

CREATE TABLE DELIVERY (
    Delivery_ID INT PRIMARY KEY,
    Order_ID INT NOT NULL,
    Delivery_Address VARCHAR(255) NOT NULL,
    Delivery_Date DATE,
    Delivery_Status VARCHAR(30) DEFAULT 'Pending',
    FOREIGN KEY (Order_ID) REFERENCES ORDERS(Order_ID)
    );
```

7.2 Sample Data — DML.

Table	No. of Records	Sample Data / Screenshot Reference
Category	3	dml.sql — Category seed records
Customer	4	dml.sql — Customer seed records
Product	4	dml.sql — Product seed records

Cart	4	dml.sql — Cart seed records
Orders	4	dml.sql — Orders seed records
Order_Item	4	dml.sql — Order_Item seed records
Payment	4	dml.sql — Payment seed records
Delivery	4	dml.sql — Delivery seed records

7.3 Constraints & Data Integrity

Constraint / Validation	Applied On	Purpose	Evidence / Screenshot
PRIMARY KEY	All eight tables	Uniquely identifies each record and prevents duplicate key values.	ddl.sql
FOREIGN KEY	Product, Cart, Orders, Order_Item, Payment, Delivery	Maintains valid relationships between related tables.	ddl.sql
NOT NULL	Required fields across tables	Prevents missing values in required attributes.	ddl.sql
UNIQUE	Customer.Email;Category. Category_Name	Prevents duplicate unique identifiers/contact values.	ddl.sql
CHECK	Price, Stock, Quantity, Amount	Restricts invalid or out-of-range values.	ddl.sql
DEFAULT	Dates and selected statuses	Provides default values when not explicitly supplied.	ddl.sql

7.4 SQL Operations & Queries

.

S.No	Operation / Query	Purpose	SQL Query / Screenshot Ref.
1	Basic SELECT — all customers	Retrieve customer records.	queries(2).sql Q1
2	Filtered SELECT	Find products above a selected price.	queries(2).sql Q2
3	LIKE filtering	Search customers by name pattern.	queries(2).sql Q3
4	UPDATE	Modify existing customer data.	queries(2).sql Q4
5	DELETE	Remove a selected record	queries(2).sql Q5
6	JOIN	Combine customer and order information	queries(2).sql Q6
7	GROUP BY	Count orders for each customer	queries(2).sql Q7

S.No	Operation / Query	Purpose	SQL Query / Screenshot Ref.
8	HAVING	Filter grouped results	queries(2).sql Q8
9	Aggregate Functions	Summarize order values.	queries(2).sql Q9
10	Subquery	Find products priced above average.	queries(2).sql Q10
11	Multi-table JOIN	Display products belonging to orders.	queries(2).sql Q11

7.5 Sample Outputs / Screenshots

Figure 1: Customer records / basic SELECT output

```
SQL Query
1  SELECT *
2  FROM Customer;
```

Result Grid

Customer_ID	Name	Email	Phone	Address
1	Ravi Kumar	ravi@gmail.com	9876543210	Rajahmundry
2	Anitha	anitha@gmail.com	9876543211	Kakinada
3	Bhavana	bhavana@gmail.com	9876543212	Visakhapatnam
4	Manoj	manoj@gmail.com	9876543213	Vijayawada

Figure 2: Filtered SELECT output

Q2 SQL Query

```
SELECT *
FROM Product
WHERE Price > 10000;
```

Result Grid

Product_ID	Product_Name	Price	Stock	Category_ID
101	Laptop	65000.0	10	1
102	Smartphone	25000.0	20	1

Figure 3: LIKE filtering output

Q3 SQL Query

SQL Query

```
1  SELECT *
2  FROM Customer
3  WHERE Name LIKE 'A%';
```

Result Grid

Customer_ID	Name	Email	Phone	Address
2	Anitha	anitha@gmail.com	9876543211	Kakinada

Figure 4: Update output

Q4 SQL Query

```
UPDATE Customer
SET Phone = '9999999999'
WHERE Customer_ID = 2;
```

Result Grid

Query OK, 1 row affected

Figure 5: Delete output

Q5 SQL Query

```
DELETE FROM Customer
WHERE Customer_ID = 4;
```

Result Grid

Query OK, 1 row affected

Figure 6: Join output

Q6 SQL Query

```
SELECT c.Name, o.Order_ID, o.Total_Amount, o.Order_Status
FROM Customer c
JOIN Orders o
ON c.Customer_ID = o.Customer_ID;
```

Result Grid

Name	Order_ID	Total_Amount	Order_Status
Ravi Kumar	1001	67500.0	Placed
Anitha	1002	25000.0	Delivered
Ravi Kumar	1003	4000.0	Placed
Bhavana	1004	1500.0	Delivered

Figure 7: GROUP BY output

Q7 SQL Query

SQL Query

```
1  SELECT Customer_ID, COUNT(Order_ID) AS Order_Count
2  FROM Orders
3  GROUP BY Customer_ID;
```

Result Grid

Customer_ID	Order_Count
1	2
2	1
3	1

Figure 8: Having output

Q8 SQL Query

```
SELECT Customer_ID, COUNT(Order_ID) AS Order_Count
FROM Orders
GROUP BY Customer_ID
HAVING COUNT(Order_ID) > 1;
```

Result Grid

Customer_ID	Order_Count
1	2

Figure 9: Aggregate Functions output

Q9 SQL Query

```
SELECT SUM(Total_Amount) AS Total,
       AVG(Total_Amount) AS Average,
       MAX(Total_Amount) AS Maximum,
       MIN(Total_Amount) AS Minimum
FROM Orders;
```

Result Grid

Total	Average	Maximum	Minimum
98000.0	24500.0	67500.0	1500.0

Figure 10: Subquery output

Q10 SQL Query

```
SQL Query
  1  SELECT *
  2  FROM Product
  3  WHERE Price > (
  4      SELECT AVG(Price)
  5      FROM Product
  6  );
```

Result Grid

Product_ID	Product_Name	Price	Stock	Category_ID
101	Laptop	65000.00	10	1
102	Smartphone	25000.00	20	1

Figure 11: Multi-table JOIN output

Q11 SQL Query

```
SELECT o.Order_ID, p.Product_Name, oi.Quantity, oi.Price
FROM Orders o
JOIN Order_Item oi ON o.Order_ID = oi.Order_ID
JOIN Product p ON oi.Product_ID = p.Product_ID;
```

Result Grid

Order_ID	Product_Name	Quantity	Price
1001	Laptop	1	65000.0
1001	Headphones	1	2500.0
1002	Smartphone	1	25000.0
1003	Headphones	1	2500.0

8. MODULES & FUNCTIONALITIES

S.No	Module Name	Functionality
1	Category Management	Organizes products into categories and stores category descriptions.
2	Customer Management	Stores customer identification, contact details, address, and account-related information.
3	Product Management	Maintains product name, price, stock, description, and category information.
4	Cart Management	Maintains shopping cart records associated with customers.
5	Order Management	Stores placed orders, order dates, total amounts, and order statuses.
6	Order Item Management	Maintains products, quantities, and prices belonging to each order.
7	Payment Management	Stores payment method, amount, date, and payment status.
8	Delivery Management	Maintains delivery address, date, and delivery status.

9. GITHUB REPOSITORY & TEAM RESPONSIBILITIES

Repository Link: https://github.com/DBMS-2026/Online-Shopping-System-DBMS

Student Name	Roll Number	Role / Responsibility	Contribution Status
A.N.V.SAI BABA	25B11AI	Wrote DDL scripts (tables, keys, constraints)	Completed
A.SANTHOSH	25B11AI060	Designed ER diagram & relational schema	Completed
T.BILVA	25B11AI133	Wrote CRUD, Join & Aggregate queries	Completed
T.HEMANTH KUMAR	25B11AIC01	Compiled documentation & screenshots	Completed

10. ENHANCEMENTS & FUTURE IMPROVEMENTS

Add a complete front-end interface for customer registration, product browsing, cart management and order placement.Introduce secure password hashing and authentication instead of storing plain-text passwords.Add product reviews and ratings, wishlist management and personalized recommendations.Add payment-gateway integration and automated delivery tracking.Add database views, stored procedures, triggers and role-based access control for advanced DBMS functionality.Improve inventory management so product stock is automatically updated when orders are placed or cancelled.

11. CONCLUSION

The Online Shopping System successfully models the main entities involved in an online shopping process using a relational database. Customer, Category, Product, Cart, Orders, Order_Item, Payment, and Delivery tables are connected using primary and foreign keys, while NOT NULL, UNIQUE, CHECK and DEFAULT constraints help maintain data integrity. SQL operations demonstrate practical DBMS concepts including DDL, DML, DQL, CRUD, JOINs, aggregate functions, GROUP BY, HAVING and subqueries. The project provides a structured foundation for managing shopping information and can be extended with a full front-end, secure authentication, payment integration and additional e-commerce features..

12. REFERENCES

1. Uploaded project DDL script: **ddl.sql**.
2. Uploaded project DML/sample-data script: dml.sql.
3. Uploaded SQL query script: queries.sql.
4. Aditya University — DBMS Capstone Project documentation template.
5. GitHub repository: Online Shopping System (project repository).

APPENDIX

A. Complete SQL Script

The final SQL implementation is supplied through the uploaded project files:

• ddl.sql — database table definitions, keys, foreign keys, and constraints.

• dml.sql — sample data for all eight tables.

• queries.sql — representative SQL operations and analytical queries.

Repository: https://github.com/hemanth2326-oss/Online-Shopping-System.git

B. Additional Screenshots / Outputs

Additional evidence can include MySQL Workbench screenshots showing table creation, sample data insertion, CRUD operations, JOIN results, aggregate functions, HAVING and subquery outputs. No execution screenshots were included in the uploaded source files, so this section is intentionally left for the team to add its actual screenshots.